Cognizant – upGrad Training Program uGE_.DOTNET FSD(Python)

Week 9 Document

Name: Adapa Krishna Prasad | **Email:** krishnaprasadadapa912@gmail.com | **Batch Name:** B1_Python | **Faculty:** Kunal

---

Date: 13/04/2026

MINI PROJECT

1. Smart Expense Tracker (CLI Based)

🎯 **Problem Statement**

You need to create a **command-line application** that helps a user track their daily expenses.

📝 **Requirements**

1. When the program runs, show a menu:
    - Add Expense
    - View All Expenses
    - Filter by Category
    - Monthly Summary
    - Exit
2. **Add Expense**
    - Ask user to enter:
        - Amount (number)
        - Category (Food, Travel, etc.)
        - Date (YYYY-MM-DD)
    - Store this data

View All Expenses

- Display all saved expenses clearly

Filter by Category

- Ask user for category
- Show only matching expenses

Monthly Summary

- Show total spending for a given month

Save all data in a file so that:

- When program restarts → old data is still available

```
import json

file = "expenses.json"

try:

    with open(file, "r") as f:

        data = json.load(f)

except:

    data = []

while True:

    print("1.Add Expense")

    print("2.View All Expenses")
```

```
print("3.Filter by Category")
print("4.Monthly Summary")
print("5.Exit")
ch = input()
if ch == "1":
    amt = float(input("Enter amount: "))
    cat = input("Enter category: ")
    dt = input("Enter date (YYYY-MM-DD): ")
    data.append({"amount": amt, "category": cat, "date": dt})
    with open(file, "w") as f:
        json.dump(data, f)
elif ch == "2":
    for i in data:
        print(i)
elif ch == "3":
    c = input("Enter category: ")
    for i in data:
        if i["category"] == c:
            print(i)
elif ch == "4":
    m = input("Enter month (YYYY-MM): ")
    total = 0
    for i in data:
        if i["date"].startswith(m):
            total += i["amount"]
    print(total)
elif ch == "5":
    break
```

2. Student Management System (OOP Based)

🎯 **Problem Statement**

Create a system to manage student records using classes.

📝 **Requirements**

1. Create a class Student with:
    - name
    - roll number

    - marks
2. Program should show menu:
    - Add Student
    - View Students
    - Search Student
    - Update Marks
    - Delete Student
    - Exit
3. **Add Student**
    - Take input from user and store student
4. **View Students**
    - Display all students
5. **Search Student**
    - Search by roll number
6. **Update Marks**
    - Modify marks of existing student
7. **Delete Student**
    - Remove student from system
8. Store data in a file so it is not lost after closing program

```
import json

file = "students.json"

class Student:

    def __init__(self, name, roll, marks):

        self.name = name

        self.roll = roll

        self.marks = marks

try:

    with open(file, "r") as f:

        data = json.load(f)

except:

    data = []

while True:

    print("1.Add Student")

    print("2.View Students")

    print("3.Search Student")

    print("4.Update Marks")

    print("5.Delete Student")

    print("6.Exit")

    ch = input()

    if ch == "1":
```

```
        name = input("Enter name: ")
        roll = input("Enter roll: ")
        marks = float(input("Enter marks: "))
        s = Student(name, roll, marks)
        data.append(s.__dict__)
        with open(file, "w") as f:
            json.dump(data, f)
    elif ch == "2":
        for i in data:
            print(i)
    elif ch == "3":
        r = input("Enter roll: ")
        for i in data:
            if i["roll"] == r:
                print(i)
    elif ch == "4":
        r = input("Enter roll: ")
        for i in data:
            if i["roll"] == r:
                i["marks"] = float(input("Enter new marks: "))
        with open(file, "w") as f:
            json.dump(data, f)
    elif ch == "5":
        r = input("Enter roll: ")
        data = [i for i in data if i["roll"] != r]
        with open(file, "w") as f:
            json.dump(data, f)
    elif ch == "6":
        break
```

3. Library Management System

🎯 **Problem Statement**

Build a system to manage books in a small library.

📝 **Requirements**

1. Show menu:

- Add Book
- Issue Book
- Return Book
- View Available Books
- Exit

2. **Add Book**
   - Input:
     - Book name
     - Author

Issue Book

- Ask user name
- Mark book as issued

Return Book

- Mark book as available again

View Available Books

- Show only books not issued

Store all book records in a file

```
import json

file = "books.json"

try:

    with open(file, "r") as f:

        data = json.load(f)

except:

    data = []

while True:

    print("1.Add Book")

    print("2.Issue Book")

    print("3.Return Book")

    print("4.View Available Books")

    print("5.Exit")

    ch = input()

    if ch == "1":

        name = input("Enter book name: ")

        author = input("Enter author: ")

        data.append({"name": name, "author": author, "issued": False, "user": ""})

        with open(file, "w") as f:

            json.dump(data, f)

    elif ch == "2":

        b = input("Enter book name: ")
```

```
        u = input("Enter user name: ")
        for i in data:
            if i["name"] == b and i["issued"] == False:
                i["issued"] = True
                i["user"] = u
        with open(file, "w") as f:
            json.dump(data, f)
    elif ch == "3":
        b = input("Enter book name: ")
        for i in data:
            if i["name"] == b and i["issued"] == True:
                i["issued"] = False
                i["user"] = ""
        with open(file, "w") as f:
            json.dump(data, f)
    elif ch == "4":
        for i in data:
            if i["issued"] == False:
                print(i)
    elif ch == "5":
        break
```

4. Log Analyzer Tool

🎯 **Problem Statement**

You are given a log file containing system logs.
You need to analyze and extract useful information.

📝 **Requirements**

1. Read a text file containing logs
2. Extract:
    - All IP addresses
    - All dates
    - All lines containing word "ERROR"
3. Count:
    - Total number of errors
    - Number of times each IP appears
4. Display results clearly
5. Save the summary into a new file

```
import re
file = "log.txt"
out = "summary.txt"
with open(file, "r") as f:
    data = f.readlines()
ips = []
dates = []
errors = []
for i in data:
    ip = re.findall(r"\d+\.\d+\.\d+\.\d+", i)
    date = re.findall(r"\d{4}-\d{2}-\d{2}", i)
    if ip:
        ips.extend(ip)
    if date:
        dates.extend(date)
    if "ERROR" in i:
        errors.append(i.strip())
ip_count = {}
for i in ips:
    if i in ip_count:
        ip_count[i] += 1
    else:
        ip_count[i] = 1
print("IP Addresses:", ips)
print("Dates:", dates)
print("Error Lines:")
for i in errors:
    print(i)
print("Total Errors:", len(errors))
print("IP Count:", ip_count)
with open(out, "w") as f:
    f.write("IP Addresses:\n")
    f.write(str(ips) + "\n\n")
```

```
    f.write("Dates:\n")
    f.write(str(dates) + "\n\n")
    f.write("Error Lines:\n")
    for i in errors:
        f.write(i + "\n")
    f.write("\nTotal Errors: " + str(len(errors)) + "\n")
    f.write("IP Count:\n")
    f.write(str(ip_count))
```

5. Task Automation Tool

🎯 **Problem Statement**

Create a tool that performs operations on files inside a folder.

📝 **Requirements**

1. Ask user for folder path
2. Show menu:
    - List all files
    - Rename files
    - Filter files by extension
    - Exit
3. **List Files**
    - Display all file names
4. **Rename Files**
    - Rename files in a pattern like:
        - file1.txt → new_1.txt

Filter Files

- Ask extension (e.g., .txt)
- Show only matching files

Print messages for each operation performed

```
import os
path = input("Enter folder path: ")
if not os.path.exists(path):
    print("Invalid Path")
else:
    while True:
        print("1.List Files")
        print("2.Rename Files")
        print("3.Filter Files by Extension")
        print("4.Exit")
        ch = input()
```

```
    if ch == "1":
        files = os.listdir(path)
        for i in files:
            print(i)
    elif ch == "2":
        files = os.listdir(path)
        c = 1
        for i in files:
            old = os.path.join(path, i)
            if os.path.isfile(old):
                new_name = "new_" + str(c) + os.path.splitext(i)[1]
                new = os.path.join(path, new_name)
                os.rename(old, new)
                print(i + " -> " + new_name)
                c += 1
    elif ch == "3":
        ext = input("Enter extension: ")
        files = os.listdir(path)
        for i in files:
            if i.endswith(ext):
                print(i)
    elif ch == "4":
        break
```

PROJECT 1: Intelligent CLI-Based “Life Operating System” (LifeOS)

Objective

Build a **command-line productivity system** that acts like your personal OS to manage:

- Tasks
- Habits
- Energy levels
- Daily logs

Features (Must Implement)

1. 🧠 User Profile System

- Take user input:
    - Name

    - Daily available hours
    - Productivity type (Morning/Night person)
- Store in file
- Load automatically on restart

2. Task Engine (Not Basic CRUD)

Each task must have:

- Title
- Priority (1–5)
- Energy Required (Low/Medium/High)
- Deadline
- Status

Add logic:

- Suggest tasks based on:
    - Current time
    - User energy type
- Example:
    - Morning person → suggest high-energy tasks early

3. Habit Tracker with Streak Logic

- Add habits (e.g., gym, reading)
- Track:
    - Daily completion
    - Streak count
    - Longest streak

Challenge:

- If a habit is missed:
    - Reset streak
    - Store history

4. Energy Simulation System (UNIQUE)

- Ask user 3 times/day:
    - "How do you feel? (1–5)"
- Based on input:
    - Recommend tasks dynamically

Example:

Energy = 2 → Suggest low-energy tasks

Energy = 5 → Suggest high-priority tasks

5. Weekly Report Generator

- Generate insights:
    - Tasks completed

    - Missed deadlines
    - Habit streaks
- Show:
    - Best day
    - Worst day

6. File Handling (Important)

- Save:
    - Tasks
    - Habits
    - Logs
- Use:
    - JSON or text files

7. Smart Search & Filter

- Search tasks by:
    - Priority
    - Deadline
    - Status

```
import json

import os

from datetime import datetime

profile_file = "profile.json"

task_file = "tasks.json"

habit_file = "habits.json"

log_file = "logs.json"

def load(file):

    try:

        with open(file, "r") as f:

            return json.load(f)

    except:

        return []

def save(file, data):

    with open(file, "w") as f:

        json.dump(data, f)

profile = load(profile_file)

tasks = load(task_file)

habits = load(habit_file)

logs = load(log_file)
```

```
if not profile:
    name = input("Enter name: ")
    hours = input("Daily hours: ")
    ptype = input("Morning/Night: ")
    profile = {"name": name, "hours": hours, "type": ptype}
    save(profile_file, profile)
def suggest_tasks(energy):
    suggested = []
    for t in tasks:
        if t["status"] != "Done":
            if energy >= 4 and t["priority"] >= 4:
                suggested.append(t)
            elif energy <= 2 and t["energy"] == "Low":
                suggested.append(t)
    return suggested
def update_streak(habit, done):
    if done:
        habit["streak"] += 1
        if habit["streak"] > habit["best"]:
            habit["best"] = habit["streak"]
    else:
        habit["streak"] = 0
while True:
    print("\n1.Add Task")
    print("2.View Tasks")
    print("3.Add Habit")
    print("4.Mark Habit")
    print("5.Energy Check")
    print("6.Weekly Report")
    print("7.Search Task")
    print("8.Exit")
    ch = input()
    if ch == "1":
```

```
    title = input("Title: ")
    priority = int(input("Priority 1-5: "))
    energy = input("Energy (Low/Medium/High): ")
    deadline = input("Deadline: ")
    tasks.append({
        "title": title,
        "priority": priority,
        "energy": energy,
        "deadline": deadline,
        "status": "Pending"
    })
    save(task_file, tasks)
elif ch == "2":
    for i in tasks:
        print(i)
elif ch == "3":
    name = input("Habit name: ")
    habits.append({
        "name": name,
        "streak": 0,
        "best": 0,
        "history": []
    })
    save(habit_file, habits)
elif ch == "4":
    h = input("Habit name: ")
    done = input("Done today? (y/n): ")
    for i in habits:
        if i["name"] == h:
            if done == "y":
                update_streak(i, True)
                i["history"].append("Done")
            else:
```

```
            update_streak(i, False)
            i["history"].append("Missed")
    save(habit_file, habits)
  elif ch == "5":
    energy = int(input("Energy (1-5): "))
    print("Suggested Tasks:")
    for i in suggest_tasks(energy):
      print(i)
  elif ch == "6":
    completed = 0
    pending = 0
    for i in tasks:
      if i["status"] == "Done":
        completed += 1
      else:
        pending += 1
    print("Completed:", completed)
    print("Pending:", pending)
    best = max(habits, key=lambda x: x["best"], default={"best": 0})
    print("Best Habit Streak:", best["best"])
  elif ch == "7":
    key = input("Search (priority/status/deadline): ")
    val = input("Value: ")
    for i in tasks:
      if str(i.get(key)) == val:
        print(i)
  elif ch == "8":
    break
```

PROJECT 2: “Digital Investigation System” (Mini Forensics Tool)

Objective

Build a **CLI-based investigation system** that analyzes logs and detects suspicious activity.

Think like a **cyber investigator** 🕵️

Scenario

You are given a log file containing:

- User logins
- File access
- Failed attempts

Your system must:

- Parse logs
- Detect anomalies
- Generate alerts

Features (Must Implement)

1. Log File Parser

- Input: text file like:

2026-04-10 10:30: user1 LOGIN SUCCESS
2026-04-10 10:35: user1 FILE_ACCESS secret.txt
2026-04-10 10:40: user2 LOGIN FAILED

Extract:

- Timestamp
- User
- Action

2. Suspicious Activity Detection

Detect:

- Multiple failed logins (e.g., >3 attempts)
- Access to restricted files
- Login at unusual hours

Example logic:
if login_time > 12 AM and < 5 AM → suspicious

3. User Activity Profiling

- For each user:
  - Total logins
  - Failed attempts
  - Files accessed

4. Risk Scoring System (UNIQUE)

Assign score:

- Failed attempts → +2
- Night login → +3
- Sensitive file → +5

Output:
user1 → Risk Score: 8 (HIGH)
user2 → Risk Score: 3 (LOW)

5. Interactive Query System

User can:

- Search logs by username
- Filter by date
- Show suspicious events

6. Report Generator

Generate:

- Top risky users
- Total suspicious events
- Timeline summary

7. File Handling

- Read logs from file
- Save reports

```
import re

import json

from datetime import datetime

file = "logs.txt"

report_file = "report.json"

try:

    with open(file, "r") as f:

        logs = f.readlines()

except:

    logs = []

users = {}

suspicious_events = []

def parse(line):

    parts = line.strip().split()

    date = parts[0]

    time = parts[1]

    user = parts[2]

    action = parts[3]
```

```
    return date, time, user, action
def add_user(user):
    if user not in users:
        users[user] = {
            "logins": 0,
            "failed": 0,
            "files": [],
            "risk": 0
        }
def is_night(hour):
    return hour >= 0 and hour <= 5
for line in logs:
    if not line.strip():
        continue
    date, time, user, action = parse(line)
    hour = int(time.split(":")[0])
    add_user(user)
    if action == "LOGIN":
        if "FAILED" in line:
            users[user]["failed"] += 1
            users[user]["risk"] += 2
            suspicious_events.append(line.strip())
        else:
            users[user]["logins"] += 1
            if is_night(hour):
                users[user]["risk"] += 3
                suspicious_events.append(line.strip())
    elif action == "FILE_ACCESS":
        file_name = line.split()[-1]
        users[user]["files"].append(file_name)
        if "secret" in file_name:
            users[user]["risk"] += 5
            suspicious_events.append(line.strip())
```

```
while True:
    print("\n1.View User Profile")
    print("2.Search by User")
    print("3.View Suspicious Events")
    print("4.Risk Report")
    print("5.Exit")
    ch = input()
    if ch == "1":
        for k, v in users.items():
            print(k, v)
    elif ch == "2":
        u = input("Enter user: ")
        print(users.get(u, "No data"))
    elif ch == "3":
        for i in suspicious_events:
            print(i)
    elif ch == "4":
        for k, v in users.items():
            level = "LOW"
            if v["risk"] >= 8:
                level = "HIGH"
            elif v["risk"] >= 4:
                level = "MEDIUM"
            print(k, "Risk Score:", v["risk"], "(", level, ")")
        report = {
            "users": users,
            "suspicious_events": suspicious_events
        }
        with open(report_file, "w") as f:
            json.dump(report, f)
    elif ch == "5":
        break
```